Hospital Information System - Mini Project Interview Question

Project Overview

Create a **Patient Management REST API** using Spring Boot that handles basic patient operations for a hospital information system.

Time Allocation

There is no time limit but we recommend not to spend more than a working day (8 hours) on it. However, in the past, this test took about 4 hours and above to complete.

Requirements

Core Functionality

Build a REST API that manages patient records with the following endpoints:

1. **POST /api/patients** - Register a new patient
2. **GET /api/patients/{id}** - Retrieve patient by ID
3. **GET /api/patients** - List all patients (with pagination)
4. **PUT /api/patients/{id}** - Update patient information
5. **DELETE /api/patients/{id}** - Soft delete a patient record

Data Model

Create a `Patient` entity with these fields:

- `id` (Long, auto-generated)
- `firstName` (String, required)
- `lastName` (String, required)
- `dateOfBirth` (Date, required)
- `phoneNumber` (String, required)
- `email` (String, optional)
- `address` (String, optional)
- `emergencyContact` (String, required)
- `bloodType` (Enum: A_POSITIVE, A_NEGATIVE, B_POSITIVE, B_NEGATIVE, AB_POSITIVE, AB_NEGATIVE, O_POSITIVE, O_NEGATIVE)
- `isActive` (Boolean, default true - for soft delete)
- `createdAt` (DateTime, auto-generated)
- `updatedAt` (DateTime, auto-updated)

Technical Requirements

Must Have:

- Input validation using Bean Validation (@Valid, @NotNull, etc.)
- Proper HTTP status codes
- Exception handling with meaningful error messages
- Basic unit tests for service layer

Nice to Have (Bonus Points):

- Pagination and sorting for the list endpoint
- Search functionality (by name or phone number)
- Swagger/OpenAPI documentation
- Integration tests
- Custom validation for phone number format
- Audit logging for patient record changes

Sample API Responses

POST /api/patients

Request:

```
{
  "firstName": "John",
  "lastName": "Doe",
  "dateOfBirth": "1985-05-15",
  "phoneNumber": "+1234567890",
  "email": "john.doe@email.com",
  "address": "123 Main St, City, State",
  "emergencyContact": "Jane Doe - +0987654321",
  "bloodType": "O_POSITIVE"
}
```

Response (201 Created):

```
{
  "id": 1,
  "firstName": "John",
  "lastName": "Doe",
  "dateOfBirth": "1985-05-15",
  "phoneNumber": "+1234567890",
  "email": "john.doe@email.com",
  "address": "123 Main St, City, State",
  "emergencyContact": "Jane Doe - +0987654321",
  "bloodType": "O_POSITIVE",
  "isActive": true,
  "createdAt": "2024-01-15T10:30:00",
  "updatedAt": "2024-01-15T10:30:00"
}
```

Submission Format

GitHub Repository Setup

1. **Create a GitHub private repository** with a descriptive name (e.g., `hospital-patient-management-api`)
2. **Share the repository** with the following GitHub users:

   adelan@examedia.com.my

3. **Push an initial commit** to the repository to indicate your start time:

bash

```
git add README.md
git commit -m "Initial commit - Project started at [timestamp]"
git push origin main
```

Development Process

1. **Commit your progress incrementally** - Push readable commits for each major feature:
   - Example commit messages:
     - `feat: add Patient entity with JPA annotations`
     - `feat: implement PatientRepository with basic CRUD operations`
     - `feat: add PatientService with business logic`
     - `feat: create PatientController REST endpoints`
     - `feat: add input validation and error handling`
     - `test: add unit tests for PatientService`
     - `docs: update README with setup instructions`
2. **Include a comprehensive README** with:
   - Setup and installation instructions
   - API documentation

Interview Presentation

Prepare for demonstration: In the follow-up interview, you will be asked to:

- **Demo the working application** (live API calls)
- **Walk through your code structure** and explain design choices